

FIPS 197

Advanced Encryption Standard (AES)

한국어번역본

NIST FIPS 197-upd1

원문발표: 2001-11-26 / Update 1: 2023-05-09

번역원칙

이문서는학습과구현검증을위한비공식한국어번역본이다. AES, Block, Byte, State, Key, Round, S-box, XOR, 함수명 (예: SUBBYTES()), 변수명, 수학적식, hexadecimal 상수등은의미가명확한기술표기로판단하여원문 English notation 을적극유지하였다. 문장설명은한국어로옮겼다. 수식과 algorithm 의의미는 NIST FIPS 197-upd1 을기준으로한다. 규격준수나법적해석이필요한경우에는 NIST 공식원문을우선한다.

Contents

서문 (Foreword)	3
초록 (Abstract)	3
AES 표준공고사항	3
1 Introduction	4
2 Definitions	4
2.1 Terms and Acronyms	4
2.2 List of Functions	5
2.3 Algorithm Parameters and Symbols	6
3 Notation and Conventions	6
3.1 Inputs and Outputs	6
3.2 Bytes	7
3.3 Indexing of Byte Sequences	7
3.4 The State	7
3.5 Arrays of Words	8
4 Mathematical Preliminaries	8
4.1 Addition in $GF(2^8)$	8
4.2 Multiplication in $GF(2^8)$	8
4.3 Multiplication of Words by a Fixed Matrix	9
4.4 Multiplicative Inverses in $GF(2^8)$	9
5 Algorithm Specifications	9
5.1 CIPHER()	10
5.1.1 SUBBYTES()	10
5.1.2 SHIFTRROWS()	11
5.1.3 MIXCOLUMNS()	11
5.1.4 ADDROUNDKEY()	11
5.2 KEYEXPANSION()	12
5.3 INVCIPHER()	12
5.3.1 INVSHIFTRROWS()	13
5.3.2 INVSUBBYTES()	13
5.3.3 INVMIXCOLUMNS()	13
5.3.4 Inverse of ADDROUNDKEY()	13
5.3.5 EQINVCIPHER()	14

6 Implementation Considerations	14
6.1 Key Length Requirements	14
6.2 Keying Restrictions	14
6.3 Parameter Extensions	14
6.4 Implementation Suggestions Regarding Various Platforms	14
6.5 Modes of Operation	15
References	15
A Key Expansion Examples	15
A.1 Expansion of a 128-bit Key	16
A.2 Expansion of a 192-bit Key	16
A.3 Expansion of a 256-bit Key	17
B Cipher Example	18
C Example Vectors	20
D Change Log (Informative)	20

서문 (Foreword)

National Institute of Standards and Technology(NIST) 의 Federal Information Processing Standards Publication Series 는 15 U.S.C. 278g-3 에따라개발되고 40 U.S.C. 11331 에따라 U.S. Secretary of Commerce 가발행하는표준및지침의공식 publication series 이다.

이 Federal Information Processing Standard publication 에관한의견은 announcement section 의 “Inquiries and Comments” 조항에제시된연락처를이용하여제출할수있다.

James A. St. Pierre, Acting Director, Information Technology Laboratory.

초록 (Abstract)

2000 년 NIST 는 Advanced Encryption Standard(AES) competition 의 winner 로 Rijndael block cipher family 를선정했다고발표하였다. Block cipher 는많은 cryptographic service 의기반이며, 특히 data confidentiality 를보장하는 service 에서핵심적으로사용된다.

이표준은 Rijndael family 중세가지 instance, 즉 AES-128, AES-192, AES-256 을규정한다. 세 algorithm 모두 data 를 128-bit block 단위로변환하며, 이름뒤의숫자는사용되는 cryptographic Key 의 bit length 를나타낸다.

Keywords: AES; block cipher; confidentiality; cryptography; encryption; Rijndael.

AES 표준공고사항

Federal Information Processing Standards Publications(FIPS) 는 NIST 가 15 U.S.C. 278g-3 에따라개발하고, U.S. Secretary of Commerce 가 40 U.S.C. 11331 에따라발행한다.

1. **Name of Standard.** Advanced Encryption Standard(AES), FIPS 197.
2. **Category of Standard.** Computer Security Standard, Cryptography.
3. **Explanation.** AES 는 electronic data 를보호하는데사용할수있는 FIPS-approved cryptographic algorithm 을규정한다. AES algorithm 은 symmetric block cipher 이며 digital information 을 encrypt(encipher) 하고 decrypt(decipher) 할수있다. AES 는 128, 192, 256-bit cryptographic Key 를사용하고 data 는 128-bit block 단위로처리한다.
4. **Approving Authority.** Secretary of Commerce.
5. **Maintenance Agency.** U.S. Department of Commerce, NIST, Information Technology Laboratory(ITL).
6. **Applicability.** FIPS 는 federal agency 가사용또는운영하는 information system, agency contractor 가운영하는 system, 또는 agency 를대신하여운영되는 system 에적용된다. 44 U.S.C. 3552 에서정의하는 national security system 에는적용되지않는다. Federal agency 는 Office of Management and Budget 및 agency policy 에따라 encryption 이적절하다고 판단한경우이표준을이용해 information 을보호할수있다. NIST 가적절하다고지정한 alternative method 를사용할수도있다. Non-Federal Government organization 도이표준을채택하여사용할수있다.
7. **Specifications.** Federal Information Processing Standard(FIPS) 197, Advanced Encryption Standard(AES).
8. **Implementations.** 이표준의 algorithm 은 software, firmware, hardware 또는이들의조합으로구현할수있다. 구체적인구현은 application, environment, technology 등에따라달라질수있다. Algorithm 은 FIPS-approved 또는 NIST-recommended mode of operation 과함께사용해야한다. AES mode 에서사용되는 Object Identifier(OID) 및관련 parameter 는 Computer Security Objects Register(CSOR) 에서확인할수있다. NIST 는 implementation 이 이표준의 algorithm 과일치하는지시험하는 validation program 을운영한다.
9. **Implementation Schedule.** 이표준은 2002 년 5 월 26 일부터효력이발생했다.
10. **Patents.** 이표준의 algorithm implementation 은미국또는외국 patent 의적용을받을수있다.

11. **Export Control.** 일부 cryptographic device 및관련 technical data 는 U.S. federal export control 대상이다. 이표준을구현한 cryptographic module 및관련 technical data 의 수출은관련법률과규정을준수하고 U.S. Department of Commerce Bureau of Industry and Security 의 license requirement 를따라야한다.
12. **Qualifications.** NIST 는 AES algorithm 분석의발전을계속추적한다. 다른 cryptographic algorithm standard 와마찬가지로이표준은 5 년마다 formal reevaluation 대상이된다. 새로운분석, 기술발전, technology breakthrough 또는 algorithm 의 mathematical weakness 가확인되면 NIST 는표준을재평가하고필요한 revision 을제공한다.
13. **Where to Obtain Copies.** 이 publication 및다른 computer security publication 은 NIST CSRC 에서무료로제공된다.
14. **Inquiries and Comments.** 이 FIPS 에대한문의와의견은 fips-197-comments@nist.gov 로제출할수있다.
15. **How to Cite This Publication.** NIST 는이 publication identifier 를 NIST FIPS 197-upd1 로지정하였다. 권장 citation 은다음과같다: National Institute of Standards and Technology (2001), *Advanced Encryption Standard (AES)*, NIST FIPS 197-upd1, updated May 9, 2023, <https://doi.org/10.6028/NIST.FIPS.197-upd1>.

1 Introduction

Block 은주어진고정길이의 bit sequence 이다. Block cipher 는 Key 라고부르는 bit sequence 에 의해 parameterized 되는 block permutation family 이다.

1997 년 NIST 는 Advanced Encryption Standard(AES) 개발작업을시작하고 block cipher candidate algorithm 의 public submission 을요청했다. Block cipher 는많은 cryptographic service 의기반이며, 특히 data confidentiality 를보장하는 service 에중요하다. 2000 년 NIST 는 AES 로 Rijndael 을선택했다.

이표준은 Rijndael 의세가지 instance 인 AES-128, AES-192, AES-256 을규정한다. 숫자는 Key 의 bit length 를나타낸다. 세경우모두 block size, 즉 data input/output length 는 128 bit 이다. Rijndael 자체는추가 block size 와 Key length 를지원하지만, 이표준에서는채택하지않는다.

이표준의구성은다음과같다.

- Section 2: terms, acronyms, algorithm parameters, symbols, functions 정의.
- Section 3: bit, byte, word 의 ordering 및 indexing notation/convention.
- Section 4: AES specification 에필요한 finite field arithmetic 및 fixed matrix multiplication.
- Section 5: AES-128, AES-192, AES-256 specification.
- Section 6: Key length, keying restriction, parameter extension, platform 별 implementation guideline.
- Appendix A: AES-128/192/256 Key expansion example.
- Appendix B: AES-128 의 step-by-step invocation example.
- Appendix C: AES example vector 를제공하는 NIST website reference.
- Appendix D: 2001 original publication 에서 2023 Update 1 으로변경된사항요약.

2 Definitions

2.1 Terms and Acronyms

용어	정의
AES	Advanced Encryption Standard.
Affine transformation	matrix multiplication 을수행한뒤 vector addition 을수행하는변환.

용어	정의
Array	고정크기의 data structure. 각 element 는 integer index 또는 indices 로식별된다.
Bit	binary digit: 0 또는 1.
Block	고정길이 bit sequence. 이표준에서 block 은 128 bit 이며, byte 또는 word array 로표현할수있다.
Block cipher	Key 에의해 parameterized 되는 block permutation family.
Byte	8 bit sequence.
Equivalent inverse cipher	CIPHER() 의 inverse 를 CIPHER() 와 유사한구조로표현한대안 specification. 수정된 key schedule 을입력으로사용한다.
Key	Block cipher family 에서사용할 permutation 을결정하는 parameter.
Key schedule	KEYEXPANSION() 이 Key 에서생성하는 Round Key sequence.
Rijndael	NIST 가 AES competition 의 winner 로선정한 block cipher.
Round	CIPHER(), INVCIPHER(), EQINVCIPHER() 에서 Nr 회반복되는 State transformation sequence. 네가지 transformation 으로구성되지만한 iteration 에서는하나가생략된다.
Round Key	Key expansion 으로부터얻는 Nr+1 개의 4-word array 중하나. AES 에서 ADDROUNDKEY() 입력으로사용된다.
State	AES block cipher 의 intermediate result. 4 행 Nb 열의 byte 2D array 로표현한다.
S-box	SUBBYTES() 와 KEYEXPANSION() 에서 byte 를 one-to-one substitution 하는 non-linear substitution table.
Word	32 bit group. 하나의 entity 또는 4-byte array 로취급한다.

2.2 List of Functions

Function	설명
ADDROUNDKEY()	Round Key 를 State 와결합하는 transformation.
AES-128()	128-bit Key 를사용하는 AES block cipher.
AES-192()	192-bit Key 를사용하는 AES block cipher.
AES-256()	256-bit Key 를사용하는 AES block cipher.
CIPHER()	AES-128/192/256 의기반이되는 block transformation. Key schedule 과 Round 수가 parameter 이다.
EQINVCIPHER()	Key schedule parameter 로 w 대신 dw 를사용하는 CIPHER() inverse.
INVCIPHER()	CIPHER() 의 inverse.
INVMIXCOLUMNS()	MIXCOLUMNS() 의 inverse.
INVSBOX()	SBOX() 의 inverse.
INVSHIFTROWS()	SHIFTROWS() 의 inverse.
INVSUBBYTES()	SUBBYTES() 의 inverse.
KEYEXPANSION()	Key 에서 Round Key 들을생성하는 routine.
KEYEXPANSIONEIC()	Equivalent inverse cipher 용수정 Round Key 를생성하는 routine.
MIXCOLUMNS()	State 의각 column 을서로독립적으로 mixing 하여새 column 을만드는 transformation.
ROTWORD()	Word 의네 byte 를 cyclic permutation 하는 transformation.
SBOX()	S-box 로정의되는 byte transformation.
SHIFTROWS()	State 의마지막세 row 를서로다른 offset 만큼 cyclic shift 하는 transformation.
SUBBYTES()	State 의각 byte 에 S-box 를독립적으로적용하는 transformation.

Function	설명
SUBWORD() XTIMES()	Word 의네 byte 각각에 S-box 를적용하는 transformation. 입력 byte 의 polynomial representation 에 x 를곱하고 $m(x)$ 로 modulo reduction 하여출력 byte 를만드는 transformation.

2.3 Algorithm Parameters and Symbols

Symbol / Parameter	설명
b^{-1}	$GF(2^8)$ element b 의 multiplicative inverse.
$\tilde{b}$	AES S-box 의 affine transformation 입력.
dw	Equivalent inverse cipher 에입력되는 key schedule 용 Word array.
GF(2)	2 개의 element 를갖는 finite field.
GF(2^8)	256 개의 element 를갖는 finite field.
in	CIPHER() 또는 INVCIPHER() 의 data input. index 0~15 의 16-byte array.
$m(x)$	Byte 를 $GF(2^8)$ element 의 polynomial 로표현할때사용하는 modulus.
key	AES-128/192/256 Key 를구성하는 N_k Word array.
Nb	State 를구성하는 column 수. 각 column 은 32-bit Word. 이표준에서 $N_b=4$.
Nk	Key 를구성하는 32-bit Word 수. AES-128/192/256 에서각각 4, 6, 8.
Nr	Round 수. AES-128/192/256 에서각각 10, 12, 14.
out	CIPHER() 또는 INVCIPHER() 의 data output. index 0~15 의 16-byte array.
Rcon	Round constant 용 Word array.
state	16 byte 의 2D array. row/column index 는각각 0~3.
$u[i]$	1D Word/Byte array u 에서 non-negative integer i 로 index 된 element.
$u[i..i+3]$	Word array 에서 $u[i], u[i+1], u[i+2], u[i+3]$ sequence.
w	Key schedule 용 Word array.
$\oplus$	Bit XOR, byte-wise XOR, 또는 word-wise XOR.
$\bullet$	$GF(2^8)$ 에서의 multiplication.
$*$	Integer multiplication.
$\leftarrow$	Pseudocode 에서 variable assignment.
{ }	Hexadecimal 또는 binary byte notation delimiter.

3 Notation and Conventions

3.1 Inputs and Outputs

Bit 는 binary digit 0 또는 1 이다. Block 은 128-bit sequence 이며 AES block cipher 의 data input 과 output 은모두 block 이다. AES block cipher 의또다른 input 인 Key 는보통미리설정되어 여러번의 block cipher invocation 동안유지된다. AES-128, AES-192, AES-256 의 Key length 는각각 128, 192, 256 bit 이다.

3.2 Bytes

AES algorithm 의 기본 processing unit 은 Byte, 즉 8-bit sequence 이다. Byte 값은 예를 들어 {10100011} 처럼 8 개의 bit 를 중괄호 안에 이어 붙여 표시할 수 있다. Byte 의 bit 를 indexed variable 로 표기할 때 이 표준에서는 왼쪽에서 오른쪽으로 index 가 감소하도록 $\{b_7b_6b_5b_4b_3b_2b_1b_0\}$ 로 쓴다.

Byte 값은 hexadecimal notation 으로 표현하는 것이 편리하다. 한 hexadecimal character 는 4 bit 를 나타낸다. Byte 는 두 hexadecimal character 로 표시하며 왼쪽 character 는 $b_7..b_4$, 오른쪽 character 는 $b_3..b_0$ 를 나타낸다. 예를 들어 {10100011} 은 hexadecimal 로 {a3} 이다.

4-bit	0000	0001	0010	0011	0100	0101	0110	0111
Hex	0	1	2	3	4	5	6	7
4-bit	1000	1001	1010	1011	1100	1101	1110	1111
Hex	8	9	a	b	c	d	e	f

3.3 Indexing of Byte Sequences

Data 와 Key input 을 byte sequence 로 모호함 없이 표현하기 위해 다음 indexing convention 을 사용한다. $8k$ 개의 bit sequence

$$r_0 \ r_1 \ r_2 \ \dots \ r_{8k-3} \ r_{8k-2} \ r_{8k-1} \quad (3.1)$$

가 주어졌을 때, $0 \leq j \leq k-1$ 에 대한 byte a_j 는

$$a_j = \{r_{8j} \ r_{8j+1} \ \dots \ r_{8j+7}\}. \quad (3.2)$$

로 정의한다. 따라서 128-bit data block $r_0 \dots r_{127}$ 은 $a_0, a_1, \dots, a_{15}$ 의 16-byte sequence 로 표현된다.

개별 Byte 내부에서는 Section 3.2 와 같이 왼쪽에서 오른쪽으로 bit index 가 7 에서 0 으로 감소한다. 반면 전체 bit sequence 에서는 첫 bit 가 index 0 부터 증가한다. 이 두 indexing 관점을 혼동하지 않는 것이 중요하다.

3.4 The State

AES block cipher 내부 연산은 **State** 라고 하는 4-by-4 Byte 2D array 에서 수행한다. State 를 s 라고 하면 각 Byte 는 row index r 와 column index c 를 가지며 $0 \leq r < 4$, $0 \leq c < 4$ 이다. 개별 Byte 는 $s_{r,c}$ 또는 $s[r, c]$ 로 쓴다.

Input array $in_0, \dots, in_{15}$ 는 다음과 같이 State 에 복사한다.

$$s[r, c] = in[r + 4c], \quad 0 \leq r < 4, \ 0 \leq c < 4. \quad (3.6)$$

모든 transformation 이 끝난 후에는 반대로

$$out[r + 4c] = s[r, c]. \quad (3.7)$$

로 output array 에 복사한다.

즉 input 이 $in_0, in_1, \dots, in_{15}$ 순서라면 State 는 **column-major** 형태로 다음과 같이 채워진다.

$$\begin{bmatrix} in_0 & in_4 & in_8 & in_{12} \\ in_1 & in_5 & in_9 & in_{13} \\ in_2 & in_6 & in_{10} & in_{14} \\ in_3 & in_7 & in_{11} & in_{15} \end{bmatrix}$$

3.5 Arrays of Words

Word 는 4 Byte sequence 이며, 하나의 128-bit Block 은 4 Word 로구성된다. State 의네 column 은각각한 Word 로해석한다. 따라서 State 의 column index c 는 Word array 의 index 가되고 row index r 는각 Word 내부 4 Byte 의 index 가된다.

1D Word array u 에서 $u[i]$ 는 index i 의 Word 를뜻하고, 연속된네 Word $u[i], u[i+1], u[i+2], u[i+3]$ 는 $u[i..i+3]$ 로표기한다.

4 Mathematical Preliminaries

Section 5 에규정된 AES transformation 중일부에서는 State 의각 Byte 를 $\mathbf{GF}(2^8)$ 라고하는 256-element finite field(Galois Field) 의한 element 로해석한다.

Byte $\{b_7b_6b_5b_4b_3b_2b_1b_0\}$ 는다음 polynomial $b(x)$ 로해석한다.

$$b(x) = b_7x^7 + b_6x^6 + b_5x^5 + b_4x^4 + b_3x^3 + b_2x^2 + b_1x + b_0. \quad (4.1)$$

예를들어 $\{01100011\}$ 은 $x^6 + x^5 + x + 1$ 이다.

4.1 Addition in $\mathbf{GF}(2^8)$

$\mathbf{GF}(2^8)$ element 두개의 addition 은 polynomial coefficient 를 modulo 2 로더하는것이다. 이는 bitwise XOR 와동일하다. 즉 $1 \oplus 1 = 0, 1 \oplus 0 = 1, 0 \oplus 0 = 0$ 이다.

예:

$$\{57\} \oplus \{83\} = \{d4\}. \quad (4.2)$$

Modulo 2 에서는 coefficient 1 과 -1 이동일하므로 addition 과 subtraction 은같은연산이다. 또한 어떤 element 를자기자신과더하면 zero element 가된다.

4.2 Multiplication in $\mathbf{GF}(2^8)$

기호 $\bullet$ 은 $\mathbf{GF}(2^8)$ multiplication 을나타낸다. 두 Byte 를 polynomial 로곱한뒤, 결과를다음 fixed polynomial 로 modulo reduction 한다.

$$m(x) = x^8 + x^4 + x^3 + x + 1. \quad (4.3)$$

따라서 Byte b, c 를나타내는 polynomial 을 $b(x), c(x)$ 라하면 $b \bullet c$ 는

$$b(x)c(x) \bmod m(x) \quad (4.4)$$

로정의된다.

특히 $\{02\}$, 즉 polynomial x 를곱하는연산은 $\mathbf{XTIMES}()$ 로표현할수있다.

$$\mathbf{XTIMES}(b) = \begin{cases} \{b_6b_5b_4b_3b_2b_1b_0\}, & b_7 = 0, \\ \{b_6b_5b_4b_3b_2b_1b_0\} \oplus \{1b\}, & b_7 = 1. \end{cases} \quad (4.5)$$

여기서 $\{1b\} = \{00011011\}$ 이다.

예를들어 $b = \{57\}$ 이면:

$$\begin{aligned} \{57\} \bullet \{01\} &= \{57\} \\ \{57\} \bullet \{02\} &= \{ae\} \\ \{57\} \bullet \{04\} &= \{47\} \\ \{57\} \bullet \{08\} &= \{8e\} \\ \{57\} \bullet \{10\} &= \{07\}. \end{aligned}$$

또한 $\{13\} = \{10\} \oplus \{02\} \oplus \{01\}$ 이므로

$$\{57\} \bullet \{13\} = \{57\} \oplus \{ae\} \oplus \{07\} = \{fe\}. \quad (4.7)$$

4.3 Multiplication of Words by a Fixed Matrix

MIXCOLUMNS() 와 INVMIXCOLUMNS() 는 fixed matrix multiplication 으로표현할수있다. Input Word $[b_0, b_1, b_2, b_3]$ 와 matrix 를정의하는 Word $[a_0, a_1, a_2, a_3]$ 가있을때 output $[d_0, d_1, d_2, d_3]$ 는다음과같다.

$$\begin{aligned} d_0 &= (a_0 \bullet b_0) \oplus (a_3 \bullet b_1) \oplus (a_2 \bullet b_2) \oplus (a_1 \bullet b_3) \\ d_1 &= (a_1 \bullet b_0) \oplus (a_0 \bullet b_1) \oplus (a_3 \bullet b_2) \oplus (a_2 \bullet b_3) \\ d_2 &= (a_2 \bullet b_0) \oplus (a_1 \bullet b_1) \oplus (a_0 \bullet b_2) \oplus (a_3 \bullet b_3) \\ d_3 &= (a_3 \bullet b_0) \oplus (a_2 \bullet b_1) \oplus (a_1 \bullet b_2) \oplus (a_0 \bullet b_3). \end{aligned} \quad (4.8)$$

4.4 Multiplicative Inverses in GF(2⁸)

Byte $b \neq \{00\}$ 에대해 multiplicative inverse b^{-1} 는

$$b \bullet b^{-1} = \{01\} \quad (4.10)$$

을만족하는유일한 Byte 이다. GF(2⁸) 에서

$$b^{-1} = b^{254} \quad (4.11)$$

로계산할수있다. 또는 extended Euclidean algorithm 을 $b(x)$ 와 $m(x)$ 에적용하여

$$b(x)a(x) + m(x)c(x) = 1 \quad (4.12)$$

을만족하는 $a(x)$ 를찾으면, $a(x)$ 가 b^{-1} 을나타낸다.

5 Algorithm Specifications

AES-128, AES-192, AES-256 을실행하는 general function 을 CIPHER() 라고하고 inverse 를 INVCIPHER() 라고한다.

CIPHER() 와 INVCIPHER() 의 core 는 State 에적용되는 fixed transformation sequence 인 **Round** 이다. 각 Round 에는추가 input 인 **Round Key** 가필요하다. Round Key 는하나의 128-bit Block 이며보통 4 Word(16 Byte) sequence 로표현한다.

KEYEXPANSION() 은 block cipher Key 를입력받아 Round Key 들을생성한다. Input Key 는 Word array key , output expanded Word array 는 w 라고하며이 w 를 **Key schedule** 이라고한다.

AES-128/192/256 은 (1) Key length, (2) Round 수와그에따라필요한 Key schedule size, (3) KEYEXPANSION() recursion rule 에서차이가난다.

Table 4: Key-Block-Round 조합

Algorithm	Key: Nk (words / bits)	Block: Nb (words / bits)	Nr
AES-128	4 / 128	4 / 128	10
AES-192	6 / 192	4 / 128	12
AES-256	8 / 256	4 / 128	14

세 algorithm 은다음과같이정의된다.

$$\begin{aligned} \text{AES-128}(in, key) &= \text{CIPHER}(in, 10, \text{KEYEXPANSION}(key)), \\ \text{AES-192}(in, key) &= \text{CIPHER}(in, 12, \text{KEYEXPANSION}(key)), \\ \text{AES-256}(in, key) &= \text{CIPHER}(in, 14, \text{KEYEXPANSION}(key)). \end{aligned} \quad (5.1)$$

Inverse permutation 은위식의 CIPHER() 를 INVCIPHER() 로바꾸어정의한다.

5.1 CIPHER()

CIPHER() 의 Round 는 State 에적용되는다음네 byte-oriented transformation 으로구성된다.

- SUBBYTES(): 각 Byte 에 S-box 를적용한다.
- SHIFTRROWS(): State row 를서로다른 offset 만큼 shift 한다.
- MIXCOLUMNS(): 각 State column 내부 data 를 mixing 한다.
- ADDROUNDKEY(): Round Key 와 State 를결합한다.

Key schedule w 는총 $4(Nr + 1)$ Word 이다.

```
procedure CIPHER(in, Nr, w)
  state <- in
  state <- ADDROUNDKEY(state, w[0..3])
  for round from 1 to Nr - 1 do
    state <- SUBBYTES(state)
    state <- SHIFTRROWS(state)
    state <- MIXCOLUMNS(state)
    state <- ADDROUNDKEY(state, w[4*round .. 4*round+3])
  end for
  state <- SUBBYTES(state)
  state <- SHIFTRROWS(state)
  state <- ADDROUNDKEY(state, w[4*Nr .. 4*Nr+3])
  return state
end procedure
```

먼저 input 을 State 로복사한뒤 initial ADDROUNDKEY() 를한번수행한다. 그다음 Round 1 부터 Nr-1 까지는네 transformation 을모두수행한다. 마지막 Round 에서는 MIXCOLUMNS() 를생략한다. 따라서 ADDROUNDKEY() 는 initial 1 회 + 각 Round 1 회, 총 Nr+1 회실행된다.

5.1.1 SUBBYTES()

SUBBYTES() 는 State 의각 Byte 에 S-box 를독립적으로적용하는 invertible, non-linear transformation 이다. AES S-box 를 SBOX() 라고한다.

Input Byte b , constant Byte $c = \{01100011\} = \{63\}$ 라하자. $b' = \text{SBOX}(b)$ 는두단계로만들어진다.

1. $\text{GF}(2^8)$ multiplicative inverse 를이용해 intermediate value $\tilde{b}$ 를만든다.

$$\tilde{b} = \begin{cases} \{00\}, & b = \{00\}, \\ b^{-1}, & b \neq \{00\}. \end{cases} \quad (5.2)$$

2. $\tilde{b}$ 의 bit 에다음 affine transformation 을적용한다.

$$b'_i = \tilde{b}_i \oplus \tilde{b}_{(i+4) \bmod 8} \oplus \tilde{b}_{(i+5) \bmod 8} \oplus \tilde{b}_{(i+6) \bmod 8} \oplus \tilde{b}_{(i+7) \bmod 8} \oplus c_i. \quad (5.3)$$

예를들어 State Byte 가 {53} 이면 S-box 의 row 5, column 3 을참조하여 {ed} 로 substitution 된다.

Table 5: SBOX(): input Byte xy 에대한 substitution value (hexadecimal)

$x \backslash y$	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
e	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

5.1.2 SHIFTRROWS()

SHIFTRROWS() 는 State 의마지막세 row 를 cyclic shift 한다. Row index r 에대해

$$s'_{r,c} = s_{r,(c+r) \bmod 4}, \quad 0 \leq r < 4, 0 \leq c < 4. \quad (5.5)$$

즉 row 0 은그대로이고, row 1 은왼쪽 1 칸, row 2 는왼쪽 2 칸, row 3 은왼쪽 3 칸 cyclic shift 된다.

$$\begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{bmatrix} \longrightarrow \begin{bmatrix} a & b & c & d \\ f & g & h & e \\ k & l & i & j \\ p & m & n & o \end{bmatrix}$$

5.1.3 MIXCOLUMNS()

MIXCOLUMNS() 는각 State column 에동일한 fixed matrix 를 $GF(2^8)$ arithmetic 으로곱한다.

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix}. \quad (5.7)$$

예를들어첫 output Byte 는

$$s'_{0,c} = (\{02\} \bullet s_{0,c}) \oplus (\{03\} \bullet s_{1,c}) \oplus s_{2,c} \oplus s_{3,c}. \quad (5.8)$$

이다. 다른세 Byte 도 matrix row 에따라같은방식으로계산한다.

5.1.4 ADDROUNDKEY()

ADDROUNDKEY() 는 State 와 Round Key 를 bitwise XOR 로결합한다. Round Key 는 Key schedule 의네 Word 이며각 Word 가 State 의한 column 과 XOR 된다.

$$[s'_{0,c}, s'_{1,c}, s'_{2,c}, s'_{3,c}] = [s_{0,c}, s_{1,c}, s_{2,c}, s_{3,c}] \oplus w[4 \cdot \text{round} + c]. \quad (5.9)$$

CIPHER() 에서 ADDROUNDKEY() 는 Round 전 initial 단계에한번, 그리고각 Round 에한번씩총 $Nr+1$ 회실행된다.

5.2 KEYEXPANSION()

KEYEXPANSION() 은 Key 에서 $4(Nr + 1)$ 개의 Word 를 생성한다. Output 은 $w[i]$, $0 \leq i < 4(Nr + 1)$ 인 linear Word array 이다.

Round constant $Rcon[j]$ 는 다음과 같다.

j	$Rcon[j]$	j	$Rcon[j]$
1	01000000	6	20000000
2	02000000	7	40000000
3	04000000	8	80000000
4	08000000	9	1b000000
5	10000000	10	36000000

Word $[a_0, a_1, a_2, a_3]$ 에 대해

$$\text{ROTWORD}([a_0, a_1, a_2, a_3]) = [a_1, a_2, a_3, a_0] \quad (5.10)$$

이고

$$\text{SUBWORD}([a_0, a_1, a_2, a_3]) = [\text{SBOX}(a_0), \text{SBOX}(a_1), \text{SBOX}(a_2), \text{SBOX}(a_3)]. \quad (5.11)$$

```

procedure KEYEXPANSION(key)
  i <- 0
  while i <= Nk - 1 do
    w[i] <- key[4*i .. 4*i+3]
    i <- i + 1
  end while
  while i <= 4*Nr + 3 do
    temp <- w[i-1]
    if i mod Nk = 0 then
      temp <- SUBWORD(ROTWORD(temp)) XOR Rcon[i/Nk]
    else if Nk > 6 and i mod Nk = 4 then
      temp <- SUBWORD(temp)
    end if
    w[i] <- w[i-Nk] XOR temp
    i <- i + 1
  end while
  return w
end procedure

```

처음 Nk 개의 Word 는 원래 Key 자체이다. 이후의 $w[i]$ 는 이전 Word $w[i - 1]$ 와 Nk 위치 이전 Word $w[i - Nk]$ 에서 recursive 하게 생성한다.

- i 가 Nk 의 multiple 이면 $w[i] = w[i - Nk] \oplus \text{SUBWORD}(\text{ROTWORD}(w[i - 1])) \oplus Rcon[i/Nk]$.
- AES-256 에서 $i \bmod 8 = 4$ 이면 $w[i] = w[i - 8] \oplus \text{SUBWORD}(w[i - 1])$.
- 나머지는 $w[i] = w[i - Nk] \oplus w[i - 1]$.

5.3 INVCIPHER()

INVCIPHER() 는 CIPHER() 의 transformation 을 inverse 로 바꾸고 reverse order 로 수행한다.

```

procedure INVCIPHER(in, Nr, w)
  state <- in
  state <- ADDROUNDKEY(state, w[4*Nr .. 4*Nr+3])
  for round from Nr - 1 downto 1 do
    state <- INVSHIFTRROWS(state)
    state <- INVSUBBYTES(state)
  end for

```

```

        state <- ADDROUNDKEY(state, w[4*round .. 4*round+3])
        state <- INVMIXCOLUMNS(state)
    end for
    state <- INVSHIFTRROWS(state)
    state <- INVSUBBYTES(state)
    state <- ADDROUNDKEY(state, w[0..3])
    return state
end procedure

```

5.3.1 INVSHIFTRROWS()

INVSHIFTRROWS() 는 SHIFTRROWS() 의 inverse 이다.

$$s'_{r,c} = s_{r,(c-r) \bmod 4} \quad (5.12)$$

즉 row 0 은 그대로, row 1/2/3 은 각각 오른쪽 1/2/3 칸 cyclic shift 한다.

5.3.2 INVSUBBYTES()

INVSUBBYTES() 는 State 의 각 Byte 에 inverse S-box 인 INVSBOX() 를 적용한다. INVSBOX() 는 S-box input/output relation 을 뒤집어 얻는다.

Table 6: INVSBOX(): input Byte xy 에 대한 inverse substitution value (hexadecimal)

$x \backslash y$	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	52	09	6a	d5	30	36	a5	38	bf	40	a3	9e	81	f3	d7	fb
1	7c	e3	39	82	9b	2f	ff	87	34	8e	43	44	c4	de	e9	cb
2	54	7b	94	32	a6	c2	23	3d	ee	4c	95	0b	42	fa	c3	4e
3	08	2e	a1	66	28	d9	24	b2	76	5b	a2	49	6d	8b	d1	25
4	72	f8	f6	64	86	68	98	16	d4	a4	5c	cc	5d	65	b6	92
5	6c	70	48	50	fd	ed	b9	da	5e	15	46	57	a7	8d	9d	84
6	90	d8	ab	00	8c	bc	d3	0a	f7	e4	58	05	b8	b3	45	06
7	d0	2c	1e	8f	ca	3f	0f	02	c1	af	bd	03	01	13	8a	6b
8	3a	91	11	41	4f	67	dc	ea	97	f2	cf	ce	f0	b4	e6	73
9	96	ac	74	22	e7	ad	35	85	e2	f9	37	e8	1c	75	df	6e
a	47	f1	1a	71	1d	29	c5	89	6f	b7	62	0e	aa	18	be	1b
b	fc	56	3e	4b	c6	d2	79	20	9a	db	c0	fe	78	cd	5a	f4
c	1f	dd	a8	33	88	07	c7	31	b1	12	10	59	27	80	ec	5f
d	60	51	7f	a9	19	b5	4a	0d	2d	e5	7a	9f	93	c9	9c	ef
e	a0	e0	3b	4d	ae	2a	f5	b0	c8	eb	bb	3c	83	53	99	61
f	17	2b	04	7e	ba	77	d6	26	e1	69	14	63	55	21	0c	7d

5.3.3 INVMIXCOLUMNS()

INVMIXCOLUMNS() 는 MIXCOLUMNS() 의 inverse 이며 다음 fixed matrix 를 각 State column 에 곱한다.

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 0e & 0b & 0d & 09 \\ 09 & 0e & 0b & 0d \\ 0d & 09 & 0e & 0b \\ 0b & 0d & 09 & 0e \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix} \quad (5.14)$$

5.3.4 Inverse of ADDROUNDKEY()

ADDROUNDKEY() 는 XOR 를 사용하므로 자기 자신이 inverse 이다. 즉 동일한 Round Key 를 다시 XOR 하면 원래 State 로 돌아간다.

5.3.5 EQINVCIPHER()

AES 의 여러 algebraic property 를 이용하면 CIPHER() 와 유사한 structure 로 inverse 를 표현할 수 있다. 이를 equivalent inverse cipher, EQINVCIPHER() 라고 한다. 이 방식에서는 modified Key schedule dw 를 사용한다.

```
procedure EQINVCIPHER(in, Nr, dw)
  state <- in
  state <- ADDROUNDKEY(state, dw[4*Nr .. 4*Nr+3])
  for round from Nr - 1 downto 1 do
    state <- INVSUBBYTES(state)
    state <- INVSHIFTRROWS(state)
    state <- INVMIXCOLUMNS(state)
    state <- ADDROUNDKEY(state, dw[4*round .. 4*round+3])
  end for
  state <- INVSUBBYTES(state)
  state <- INVSHIFTRROWS(state)
  state <- ADDROUNDKEY(state, dw[0..3])
  return state
end procedure
```

KEYEXPANSIONEIC() 는 먼저 일반 KEYEXPANSION() 과 같은 w 를 만든 다음 첫 번째와 마지막 Round Key 를 제외한 중간 Round Key 에 INVMIXCOLUMNS() 를 적용하여 dw 를 만든다. 첫 번째와 마지막 Round Key 는 w 와 dw 가 동일하다.

6 Implementation Considerations

6.1 Key Length Requirements

AES implementation 은 Section 5 에서 지정한 세 Key length, 즉 128, 192, 256 bit 중 적어도 하나를 지원해야 한다. 두 개 또는 세 개 모두를 optional 로 지원할 수 있으며, 이는 implementation 간 interoperability 에 도움이 될 수 있다.

6.2 Keying Restrictions

Cryptographic Key 가 적절하게 생성되었다면 (NIST SP 800-133 Rev.2 의 guideline 참조), 그 Key 를 AES algorithm 에 사용할 때 이 표준이 추가적인 제한을 두지는 않는다.

6.3 Parameter Extensions

Table 3 은 Key length(N_k), block size(N_b), Round 수 (N_r) 의 allowed value 를 명시적으로 정의한다. 향후 revision 에서 allowed parameter value 가 변경 또는 추가될 가능성이 있으므로, implementer 는 future flexibility 를 고려하여 AES implementation 을 설계할 수 있다.

6.4 Implementation Suggestions Regarding Various Platforms

같은 input Key 와 data(plaintext 또는 ciphertext) 에 대해 이 표준이 규정한 algorithm 과 동일한 output(ciphertext 또는 plaintext) 을 생성한다면, 내부 implementation 방식이 달라도 equivalent AES implementation 으로 볼 수 있다.

AES proposal document 와 NIST AES resource 는 다양한 platform 에서 efficient implementation 을 수행하는 방법을 제안한다. 다만 performance-oriented implementation suggestion 은 AES 내부 동작을 설명하기 위한 것이며 implementation attack 까지 자동으로 방어해주는 것은 아니다.

Physical implementation 은 computation time, cache behavior, fault injection 등 side channel 을통해 Key-dependent information 을 leak 할수있다. Non-invasive attack 도 physical tamper detection mechanism 이존재하더라도유�효할수있다. 예를들어 software platform 에서 main memory access 를빠르게하기위해 cache 를사용하는 AES implementation 은 cache-timing attack 의영향을받을수있다.

따라서 applicable 한환경에서는 implementation attack 에대한 protection 을고려해야한다. 이러한 protection technique 자체는 FIPS 197 의 scope 밖이지만, NIST validation program 에서 는 conformance test 와관련하여고려된다.

6.5 Modes of Operation

Block cipher mode of operation 은 block cipher 를이용하여 confidentiality, authentication 등의 information service 를제공하는 cryptographic function 이다. NIST-recommended mode 는 NIST SP 800-38 series 에규정되어있다.

References

아래 reference 의 title, author, publication identifier, URL 은원문표기를유지한다.

- [1] James Nechvatal et al., *Report on the Development of the Advanced Encryption Standard (AES)*, Journal of Research of NIST, May 2001.
- [2] Joan Daemen and Vincent Rijmen, *AES Proposal: Rijndael Document Version 2*, AES Algorithm Submission, September 1999.
- [3] Joan Daemen and Vincent Rijmen, *The Design of Rijndael - The Advanced Encryption Standard (AES)*, Second Edition, Springer, 2020.
- [4] Michael Artin, *Algebra*, Pearson Modern Classic, 2nd ed., 2017.
- [5] Alfred J. Menezes, Scott A. Vanstone, Paul C. Van Oorschot, *Handbook of Applied Cryptography*, CRC Press, 1997.
- [6] Elaine Barker, Allen Roginsky, Richard Davis, *Recommendation for Cryptographic Key Generation*, NIST SP 800-133 Rev.2, June 2020.
- [7] NIST, *AES Development*, 2022.
- [8] NIST, *Cryptographic Standards and Guidelines: Examples with Intermediate Values*, 2022.
- [9] NIST, *Crypto Publications Review Board*, 2022.
- [10] Nicky Mouha, *Review of the Advanced Encryption Standard*, NIST IR 8319.

A Key Expansion Examples

이 appendix 는각 Key size 에대한 Key schedule 전개과정을보여준다. Multi-byte value 는 Section 3 notation 을사용하며, index i 를제외한모든값은 hexadecimal 이다.

표의열의미는다음과같다.

- **temp**: 기본적으로 $w[i - 1]$.
- **ROTWORD**: $i \bmod Nk = 0$ 일때 temp 를 1 Byte left rotation 한값.
- **SUBWORD**: S-box 를각 Byte 에적용한값. AES-256 에서는 $i \bmod 8 = 4$ 인경우에도적용된다.
- **Rcon**: 해당 iteration 에서 XOR 되는 Round constant.
- **변환후 temp**: 위조건적용이끝난후의 temp.
- $w[i - Nk]$: Nk Word 이전값.
- $w[i]$: 최종적으로 $w[i - Nk] \oplus temp$ 로생성된 Word.

A.1 Expansion of a 128-bit Key

Key = 2b 7e 15 16 28 ae d2 a6 ab f7 15 88 09 cf 4f 3c. 여기서 $N_k = 4$, $N_r = 10$ 이다. 처음 4 개의 Word 는 Key 자체이다.

$w_0=2b7e1516$ $w_1=28aed2a6$ $w_2=abf71588$ $w_3=09cf4f3c$

AES-128 Key expansion intermediate values

i	temp	ROTWORD	SUBWORD	Rcon	변환후 temp	$w[i - N_k]$	$w[i]$
4	09cf4f3c	cf4f3c09	8a84eb01	01000000	8b84eb01	2b7e1516	a0fafe17
5	a0fafe17				a0fafe17	28aed2a6	88542cb1
6	88542cb1				88542cb1	abf71588	23a33939
7	23a33939				23a33939	09cf4f3c	2a6c7605
8	2a6c7605	6c76052a	50386be5	02000000	52386be5	a0fafe17	f2c295f2
9	f2c295f2				f2c295f2	88542cb1	7a96b943
10	7a96b943				7a96b943	23a33939	5935807a
11	5935807a				5935807a	2a6c7605	7359f67f
12	7359f67f	59f67f73	cb42d28f	04000000	cf42d28f	f2c295f2	3d80477d
13	3d80477d				3d80477d	7a96b943	4716fe3e
14	4716fe3e				4716fe3e	5935807a	1e237e44
15	1e237e44				1e237e44	7359f67f	6d7a883b
16	6d7a883b	7a883b6d	dac4e23c	08000000	d2c4e23c	3d80477d	ef44a541
17	ef44a541				ef44a541	4716fe3e	a8525b7f
18	a8525b7f				a8525b7f	1e237e44	b671253b
19	b671253b				b671253b	6d7a883b	db0bad00
20	db0bad00	0bad00db	2b9563b9	10000000	3b9563b9	ef44a541	d4d1c6f8
21	d4d1c6f8				d4d1c6f8	a8525b7f	7c839d87
22	7c839d87				7c839d87	b671253b	caf2b8bc
23	caf2b8bc				caf2b8bc	db0bad00	11f915bc
24	11f915bc	f915bc11	99596582	20000000	b9596582	d4d1c6f8	6d88a37a
25	6d88a37a				6d88a37a	7c839d87	110b3efd
26	110b3efd				110b3efd	caf2b8bc	dbf98641
27	dbf98641				dbf98641	11f915bc	ca0093fd
28	ca0093fd	0093fdca	63dc5474	40000000	23dc5474	6d88a37a	4e54f70e
29	4e54f70e				4e54f70e	110b3efd	5f5fc9f3
30	5f5fc9f3				5f5fc9f3	dbf98641	84a64fb2
31	84a64fb2				84a64fb2	ca0093fd	4ea6dc4f
32	4ea6dc4f	a6dc4f4e	2486842f	80000000	a486842f	4e54f70e	ead27321
33	ead27321				ead27321	5f5fc9f3	b58dbad2
34	b58dbad2				b58dbad2	84a64fb2	312bf560
35	312bf560				312bf560	4ea6dc4f	7f8d292f
36	7f8d292f	8d292f7f	5da515d2	1b000000	46a515d2	ead27321	ac7766f3
37	ac7766f3				ac7766f3	b58dbad2	19fadc21
38	19fadc21				19fadc21	312bf560	28d12941
39	28d12941				28d12941	7f8d292f	575c006e
40	575c006e	5c006e57	4a639f5b	36000000	7c639f5b	ac7766f3	d014f9a8
41	d014f9a8				d014f9a8	19fadc21	c9ee2589
42	c9ee2589				c9ee2589	28d12941	e13f0cc8
43	e13f0cc8				e13f0cc8	575c006e	b6630ca6

A.2 Expansion of a 192-bit Key

Key = 8e 73 b0 f7 da 0e 64 52 c8 10 f3 2b 80 90 79 e5 62 f8 ea d2 52 2c 6b 7b. 여기서 $N_k = 6$, $N_r = 12$ 이다. 처음 6 개의 Word 는 Key 자체이다.

$w_0=8e73b0f7$ $w_1=da0e6452$ $w_2=c810f32b$ $w_3=809079e5$ $w_4=62f8ead2$ $w_5=522c6b7b$

AES-192 Key expansion intermediate values

i	temp	ROTWORD	SUBWORD	Rcon	변환후 temp	$w[i - N_k]$	$w[i]$
6	522c6b7b	2c6b7b52	717f2100	01000000	707f2100	8e73b0f7	fe0c91f7
7	fe0c91f7				fe0c91f7	da0e6452	2402f5a5
8	2402f5a5				2402f5a5	c810f32b	ec12068e

i	temp	ROTWORD	SUBWORD	Rcon	변환후 temp	$w[i - N_k]$	$w[i]$
9	ec12068e				ec12068e	809079e5	6c827f6b
10	6c827f6b				6c827f6b	62f8ead2	0e7a95b9
11	0e7a95b9				0e7a95b9	522c6b7b	5c56fec2
12	5c56fec2	56fec25c	b1bb254a	02000000	b3bb254a	fe0c91f7	4db7b4bd
13	4db7b4bd				4db7b4bd	2402f5a5	69b54118
14	69b54118				69b54118	ec12068e	85a74796
15	85a74796				85a74796	6c827f6b	e92538fd
16	e92538fd				e92538fd	0e7a95b9	e75fad44
17	e75fad44				e75fad44	5c56fec2	bb095386
18	bb095386	095386bb	01ed44ea	04000000	05ed44ea	4db7b4bd	485af057
19	485af057				485af057	69b54118	21efb14f
20	21efb14f				21efb14f	85a74796	a448f6d9
21	a448f6d9				a448f6d9	e92538fd	4d6dce24
22	4d6dce24				4d6dce24	e75fad44	aa326360
23	aa326360				aa326360	bb095386	113b30e6
24	113b30e6	3b30e611	e2048e82	08000000	ea048e82	485af057	a25e7ed5
25	a25e7ed5				a25e7ed5	21efb14f	83b1cf9a
26	83b1cf9a				83b1cf9a	a448f6d9	27f93943
27	27f93943				27f93943	4d6dce24	6a94f767
28	6a94f767				6a94f767	aa326360	c0a69407
29	c0a69407				c0a69407	113b30e6	d19da4e1
30	d19da4e1	9da4e1d1	5e49f83e	10000000	4e49f83e	a25e7ed5	ec1786eb
31	ec1786eb				ec1786eb	83b1cf9a	6fa64971
32	6fa64971				6fa64971	27f93943	485f7032
33	485f7032				485f7032	6a94f767	22cb8755
34	22cb8755				22cb8755	c0a69407	e26d1352
35	e26d1352				e26d1352	d19da4e1	33f0b7b3
36	33f0b7b3	f0b7b333	8ca96dc3	20000000	aca96dc3	ec1786eb	40beeb28
37	40beeb28				40beeb28	6fa64971	2f18a259
38	2f18a259				2f18a259	485f7032	6747d26b
39	6747d26b				6747d26b	22cb8755	458c553e
40	458c553e				458c553e	e26d1352	a7e1466c
41	a7e1466c				a7e1466c	33f0b7b3	9411f1df
42	9411f1df	11f1df94	82a19e22	40000000	c2a19e22	40beeb28	821f750a
43	821f750a				821f750a	2f18a259	ad07d753
44	ad07d753				ad07d753	6747d26b	ca400538
45	ca400538				ca400538	458c553e	8fcc5006
46	8fcc5006				8fcc5006	a7e1466c	282d166a
47	282d166a				282d166a	9411f1df	bc3ce7b5
48	bc3ce7b5	3ce7b5bc	eb94d565	80000000	6b94d565	821f750a	e98ba06f
49	e98ba06f				e98ba06f	ad07d753	448c773c
50	448c773c				448c773c	ca400538	8ecc7204
51	8ecc7204				8ecc7204	8fcc5006	01002202

A.3 Expansion of a 256-bit Key

Key = 60 3d eb 10 15 ca 71 be 2b 73 ae f0 85 7d 77 81 1f 35 2c 07 3b 61 08 d7 2d 98 10 a3 09 14 df f4. 여기서 $N_k = 8$, $N_r = 14$ 이다. 처음 8 개의 Word 는 Key 자체이다.

$w_0=603deb10$ $w_1=15ca71be$ $w_2=2b73aef0$ $w_3=857d7781$
 $w_4=1f352c07$ $w_5=3b6108d7$ $w_6=2d9810a3$ $w_7=0914dff4$

AES-256 Key expansion intermediate values

i	temp	ROTWORD	SUBWORD	Rcon	변환후 temp	$w[i - N_k]$	$w[i]$
8	0914dff4	14dff409	fa9ebf01	01000000	fb9ebf01	603deb10	9ba35411
9	9ba35411				9ba35411	15ca71be	8e6925af
10	8e6925af				8e6925af	2b73aef0	a51a8b5f
11	a51a8b5f				a51a8b5f	857d7781	2067fcde
12	2067fcde		b785b01d		b785b01d	1f352c07	a8b09c1a
13	a8b09c1a				a8b09c1a	3b6108d7	93d194cd
14	93d194cd				93d194cd	2d9810a3	be49846e
15	be49846e				be49846e	0914dff4	b75d5b9a
16	b75d5b9a	5d5b9ab7	4c39b8a9	02000000	4e39b8a9	9ba35411	d59aecb8
17	d59aecb8				d59aecb8	8e6925af	5bf3c917
18	5bf3c917				5bf3c917	a51a8b5f	fee94248

i	temp	ROTWORD	SUBWORD	Rcon	변환후 temp	$w[i - N_k]$	$w[i]$
19	fee94248				fee94248	2067fcde	de8ebe96
20	de8ebe96		1d19ae90		1d19ae90	a8b09c1a	b5a9328a
21	b5a9328a				b5a9328a	93d194cd	2678a647
22	2678a647				2678a647	be49846e	98312229
23	98312229				98312229	b75d5b9a	2f6c79b3
24	2f6c79b3	6c79b32f	50b66d15	04000000	54b66d15	d59aecb8	812c81ad
25	812c81ad				812c81ad	5bf3c917	dadf48ba
26	dadf48ba				dadf48ba	fee94248	24360af2
27	24360af2				24360af2	de8ebe96	fab8b464
28	fab8b464		2d6c8d43		2d6c8d43	b5a9328a	98c5bfc9
29	98c5bfc9				98c5bfc9	2678a647	bebd198e
30	bebd198e				bebd198e	98312229	268c3ba7
31	268c3ba7				268c3ba7	2f6c79b3	09e04214
32	09e04214	e0421409	e12cfa01	08000000	e92cfa01	812c81ad	68007bac
33	68007bac				68007bac	dadf48ba	b2df3316
34	b2df3316				b2df3316	24360af2	96e939e4
35	96e939e4				96e939e4	fab8b464	6c518d80
36	6c518d80		50d15dcd		50d15dcd	98c5bfc9	c814e204
37	c814e204				c814e204	bebd198e	76a9fb8a
38	76a9fb8a				76a9fb8a	268c3ba7	5025c02d
39	5025c02d				5025c02d	09e04214	59c58239
40	59c58239	c5823959	a61312cb	10000000	b61312cb	68007bac	de136967
41	de136967				de136967	b2df3316	6ccc5a71
42	6ccc5a71				6ccc5a71	96e939e4	fa256395
43	fa256395				fa256395	6c518d80	9674ee15
44	9674ee15		90922859		90922859	c814e204	5886ca5d
45	5886ca5d				5886ca5d	76a9fb8a	2e2f31d7
46	2e2f31d7				2e2f31d7	5025c02d	7e0af1fa
47	7e0af1fa				7e0af1fa	59c58239	27cf73c3
48	27cf73c3	cf73c327	8a8f2ecc	20000000	aa8f2ecc	de136967	749c47ab
49	749c47ab				749c47ab	6ccc5a71	18501dda
50	18501dda				18501dda	fa256395	e2757e4f
51	e2757e4f				e2757e4f	9674ee15	7401905a
52	7401905a		927c60be		927c60be	5886ca5d	cafaaae3
53	cafaaae3				cafaaae3	2e2f31d7	e4d59b34
54	e4d59b34				e4d59b34	7e0af1fa	9adf6ace
55	9adf6ace				9adf6ace	27cf73c3	bd10190d
56	bd10190d	10190dbd	cad4d77a	40000000	8ad4d77a	749c47ab	fe4890d1
57	fe4890d1				fe4890d1	18501dda	e6188d0b
58	e6188d0b				e6188d0b	e2757e4f	046df344
59	046df344				046df344	7401905a	706c631e

B Cipher Example

이 example 은 block length 와 Key length 가 모두 16 Byte, 즉 $N_b=4$, $N_k=4$ 인 AES-128 에서 State 가 Round 별로 어떻게 변하는지를 보여준다.

Input = 32 43 f6 a8 88 5a 30 8d 31 31 98 a2 e0 37 07 34

Key = 2b 7e 15 16 28 ae d2 a6 ab f7 15 88 09 cf 4f 3c

Round Key 는 Appendix A 의 AES-128 Key Expansion 결과를 사용한다. 아래 matrix 는 FIPS 197 의 State ordering, 즉 column-major mapping 을 따른다.

Round 1

Start of Round	After SUBBYTES	After SHIFTRWS	After MIXCOLUMNS	Round Key
$\begin{bmatrix} 19 & a0 & 9a & e9 \\ 3d & f4 & c6 & f8 \\ e3 & e2 & 8d & 48 \\ be & 2b & 2a & 08 \end{bmatrix}$	$\begin{bmatrix} d4 & e0 & b8 & 1e \\ 27 & bf & b4 & 41 \\ 11 & 98 & 5d & 52 \\ ae & f1 & e5 & 30 \end{bmatrix}$	$\begin{bmatrix} d4 & e0 & b8 & 1e \\ bf & b4 & 41 & 27 \\ 5d & 52 & 11 & 98 \\ 30 & ae & f1 & e5 \end{bmatrix}$	$\begin{bmatrix} 04 & e0 & 48 & 28 \\ 66 & cb & f8 & 06 \\ 81 & 19 & d3 & 26 \\ e5 & 9a & 7a & 4c \end{bmatrix}$	$\begin{bmatrix} a0 & 88 & 23 & 2a \\ fa & 54 & a3 & 6c \\ fe & 2c & 39 & 76 \\ 17 & b1 & 39 & 05 \end{bmatrix}$

Round 2

Start of Round	After SUBBYTES	After SHIFTRROWS	After MIXCOLUMNS	Round Key
$\begin{bmatrix} a4 & 68 & 6b & 02 \\ 9c & 9f & 5b & 6a \\ 7f & 35 & ea & 50 \\ f2 & 2b & 43 & 49 \end{bmatrix}$	$\begin{bmatrix} 49 & 45 & 7f & 77 \\ de & db & 39 & 02 \\ d2 & 96 & 87 & 53 \\ 89 & f1 & 1a & 3b \end{bmatrix}$	$\begin{bmatrix} 49 & 45 & 7f & 77 \\ db & 39 & 02 & de \\ 87 & 53 & d2 & 96 \\ 3b & 89 & f1 & 1a \end{bmatrix}$	$\begin{bmatrix} 58 & 1b & db & 1b \\ 4d & 4b & e7 & 6b \\ ca & 5a & ca & b0 \\ f1 & ac & a8 & e5 \end{bmatrix}$	$\begin{bmatrix} f2 & 7a & 59 & 73 \\ c2 & 96 & 35 & 59 \\ 95 & b9 & 80 & f6 \\ f2 & 43 & 7a & 7f \end{bmatrix}$

Round 3

Start of Round	After SUBBYTES	After SHIFTRROWS	After MIXCOLUMNS	Round Key
$\begin{bmatrix} aa & 61 & 82 & 68 \\ 8f & dd & d2 & 32 \\ 5f & e3 & 4a & 46 \\ 03 & ef & d2 & 9a \end{bmatrix}$	$\begin{bmatrix} ac & ef & 13 & 45 \\ 73 & c1 & b5 & 23 \\ cf & 11 & d6 & 5a \\ 7b & df & b5 & b8 \end{bmatrix}$	$\begin{bmatrix} ac & ef & 13 & 45 \\ c1 & b5 & 23 & 73 \\ d6 & 5a & cf & 11 \\ b8 & 7b & df & b5 \end{bmatrix}$	$\begin{bmatrix} 75 & 20 & 53 & bb \\ ec & 0b & c0 & 25 \\ 09 & 63 & cf & d0 \\ 93 & 33 & 7c & dc \end{bmatrix}$	$\begin{bmatrix} 3d & 47 & 1e & 6d \\ 80 & 16 & 23 & 7a \\ 47 & fe & 7e & 88 \\ 7d & 3e & 44 & 3b \end{bmatrix}$

Round 4

Start of Round	After SUBBYTES	After SHIFTRROWS	After MIXCOLUMNS	Round Key
$\begin{bmatrix} 48 & 67 & 4d & d6 \\ 6c & 1d & e3 & 5f \\ 4e & 9d & b1 & 58 \\ ee & 0d & 38 & e7 \end{bmatrix}$	$\begin{bmatrix} 52 & 85 & e3 & f6 \\ 50 & a4 & 11 & cf \\ 2f & 5e & c8 & 6a \\ 28 & d7 & 07 & 94 \end{bmatrix}$	$\begin{bmatrix} 52 & 85 & e3 & f6 \\ a4 & 11 & cf & 50 \\ c8 & 6a & 2f & 5e \\ 94 & 28 & d7 & 07 \end{bmatrix}$	$\begin{bmatrix} 0f & 60 & 6f & 5e \\ d6 & 31 & c0 & b3 \\ da & 38 & 10 & 13 \\ a9 & bf & 6b & 01 \end{bmatrix}$	$\begin{bmatrix} ef & a8 & b6 & db \\ 44 & 52 & 71 & 0b \\ a5 & 5b & 25 & ad \\ 41 & 7f & 3b & 00 \end{bmatrix}$

Round 5

Start of Round	After SUBBYTES	After SHIFTRROWS	After MIXCOLUMNS	Round Key
$\begin{bmatrix} e0 & c8 & d9 & 85 \\ 92 & 63 & b1 & b8 \\ 7f & 63 & 35 & be \\ e8 & c0 & 50 & 01 \end{bmatrix}$	$\begin{bmatrix} e1 & e8 & 35 & 97 \\ 4f & fb & c8 & 6c \\ d2 & fb & 96 & ae \\ 9b & ba & 53 & 7c \end{bmatrix}$	$\begin{bmatrix} e1 & e8 & 35 & 97 \\ fb & c8 & 6c & 4f \\ 96 & ae & d2 & fb \\ 7c & 9b & ba & 53 \end{bmatrix}$	$\begin{bmatrix} 25 & bd & b6 & 4c \\ d1 & 11 & 3a & 4c \\ a9 & d1 & 33 & c0 \\ ad & 68 & 8e & b0 \end{bmatrix}$	$\begin{bmatrix} d4 & 7c & ca & 11 \\ d1 & 83 & f2 & f9 \\ c6 & 9d & b8 & 15 \\ f8 & 87 & bc & bc \end{bmatrix}$

Round 6

Start of Round	After SUBBYTES	After SHIFTRROWS	After MIXCOLUMNS	Round Key
$\begin{bmatrix} f1 & c1 & 7c & 5d \\ 00 & 92 & c8 & b5 \\ 6f & 4c & 8b & d5 \\ 55 & ef & 32 & 0c \end{bmatrix}$	$\begin{bmatrix} a1 & 78 & 10 & 4c \\ 63 & 4f & e8 & d5 \\ a8 & 29 & 3d & 03 \\ fc & df & 23 & fe \end{bmatrix}$	$\begin{bmatrix} a1 & 78 & 10 & 4c \\ 4f & e8 & d5 & 63 \\ 3d & 03 & a8 & 29 \\ fe & fc & df & 23 \end{bmatrix}$	$\begin{bmatrix} 4b & 2c & 33 & 37 \\ 86 & 4a & 9d & d2 \\ 8d & 89 & f4 & 18 \\ 6d & 80 & e8 & d8 \end{bmatrix}$	$\begin{bmatrix} 6d & 11 & db & ca \\ 88 & 0b & f9 & 00 \\ a3 & 3e & 86 & 93 \\ 7a & fd & 41 & fd \end{bmatrix}$

Round 7

Start of Round	After SUBBYTES	After SHIFTRROWS	After MIXCOLUMNS	Round Key
$\begin{bmatrix} 26 & 3d & e8 & fd \\ 0e & 41 & 64 & d2 \\ 2e & b7 & 72 & 8b \\ 17 & 7d & a9 & 25 \end{bmatrix}$	$\begin{bmatrix} f7 & 27 & 9b & 54 \\ ab & 83 & 43 & b5 \\ 31 & a9 & 40 & 3d \\ f0 & ff & d3 & 3f \end{bmatrix}$	$\begin{bmatrix} f7 & 27 & 9b & 54 \\ 83 & 43 & b5 & ab \\ 40 & 3d & 31 & a9 \\ 3f & f0 & ff & d3 \end{bmatrix}$	$\begin{bmatrix} 14 & 46 & 27 & 34 \\ 15 & 16 & 46 & 2a \\ b5 & 15 & 56 & d8 \\ bf & ec & d7 & 43 \end{bmatrix}$	$\begin{bmatrix} 4e & 5f & 84 & 4e \\ 54 & 5f & a6 & a6 \\ f7 & c9 & 4f & dc \\ 0e & f3 & b2 & 4f \end{bmatrix}$

Round 8

Start of Round	After SUBBYTES	After SHIFTRROWS	After MIXCOLUMNS	Round Key
$\begin{bmatrix} 5a & 19 & a3 & 7a \\ 41 & 49 & e0 & 8c \\ 42 & dc & 19 & 04 \\ b1 & 1f & 65 & 0c \end{bmatrix}$	$\begin{bmatrix} be & d4 & 0a & da \\ 83 & 3b & e1 & 64 \\ 2c & 86 & d4 & f2 \\ c8 & c0 & 4d & fe \end{bmatrix}$	$\begin{bmatrix} be & d4 & 0a & da \\ 3b & e1 & 64 & 83 \\ d4 & f2 & 2c & 86 \\ fe & c8 & c0 & 4d \end{bmatrix}$	$\begin{bmatrix} 00 & b1 & 54 & fa \\ 51 & c8 & 76 & 1b \\ 2f & 89 & 6d & 99 \\ d1 & ff & cd & ea \end{bmatrix}$	$\begin{bmatrix} ea & b5 & 31 & 7f \\ d2 & 8d & 2b & 8d \\ 73 & ba & f5 & 29 \\ 21 & d2 & 60 & 2f \end{bmatrix}$

Round 9

Start of Round	After SUBBYTES	After SHIFTRWS	After MIXCOLUMNS	Round Key
$\begin{bmatrix} \text{ea} & 04 & 65 & 85 \\ 83 & 45 & 5d & 96 \\ 5c & 33 & 98 & b0 \\ f0 & 2d & ad & c5 \end{bmatrix}$	$\begin{bmatrix} 87 & f2 & 4d & 97 \\ ec & 6e & 4c & 90 \\ 4a & c3 & 46 & e7 \\ 8c & d8 & 95 & a6 \end{bmatrix}$	$\begin{bmatrix} 87 & f2 & 4d & 97 \\ 6e & 4c & 90 & ec \\ 46 & e7 & 4a & c3 \\ a6 & 8c & d8 & 95 \end{bmatrix}$	$\begin{bmatrix} 47 & 40 & a3 & 4c \\ 37 & d4 & 70 & 9f \\ 94 & e4 & 3a & 42 \\ ed & a5 & a6 & bc \end{bmatrix}$	$\begin{bmatrix} ac & 19 & 28 & 57 \\ 77 & fa & d1 & 5c \\ 66 & dc & 29 & 00 \\ f3 & 21 & 41 & 6e \end{bmatrix}$

Round 10

Start of Round	After SUBBYTES	After SHIFTRWS	Round Key
$\begin{bmatrix} eb & 59 & 8b & 1b \\ 40 & 2e & a1 & c3 \\ f2 & 38 & 13 & 42 \\ 1e & 84 & e7 & d2 \end{bmatrix}$	$\begin{bmatrix} e9 & cb & 3d & af \\ 09 & 31 & 32 & 2e \\ 89 & 07 & 7d & 2c \\ 72 & 5f & 94 & b5 \end{bmatrix}$	$\begin{bmatrix} e9 & cb & 3d & af \\ 31 & 32 & 2e & 09 \\ 7d & 2c & 89 & 07 \\ b5 & 72 & 5f & 94 \end{bmatrix}$	$\begin{bmatrix} d0 & c9 & e1 & b6 \\ 14 & ee & 3f & 63 \\ f9 & 25 & 0c & 0c \\ a8 & 89 & c8 & a6 \end{bmatrix}$

Output = 39 25 84 1d 02 dc 09 fb dc 11 85 97 19 6a 0b 32.

C Example Vectors

NIST Computer Security Resource Center 는 AES 에대한 “examples with intermediate values” website 를제공한다. AES-128, AES-192, AES-256 의 extensive example vector 는 NIST Cryptographic Standards and Guidelines 의 example values resource 에서확인할수 있다.

D Change Log (Informative)

2001 년 11 월 26 일의 original FIPS 197 은 NIST Crypto Publication Review Board 의 review 를거쳐 update 되었다. Public comment 와 NIST IR 8319 의 AES security analysis 를바탕으로 AES 의 technical specification 자체는유지하기로결정했다.

2023 년 5 월 9 일 Update 1 의주요 editorial change 는다음과같다.

1. Publication 의여러 formatting element 를개선하고문장을 clarity 를위해수정했다.
2. Front matter 에 title page, foreword, abstract, keywords 를추가했다. Title page 의 official name 과 affiliation 은 original publication 당시정보를반영한다.
3. Announcement section 을현재 statute, regulation, standard, guideline, validation program 에맞게 update 했다.
4. Section 1 을수정하여 AES development effort reference 를추가/갱신하고 AES-128, AES-192, AES-256 을명시적으로이름붙였다.
5. 기존 Section 2.2 의 Algorithm Parameters, Symbols and Functions 를새로운 Section 2.2 List of Functions 와 Section 2.3 Algorithm Parameters and Symbols 로분리했다.
6. Specification 에서사용되는 term, function, symbol 을 Sections 2.1–2.3 에보다 comprehensive 하게포함했다.
7. Indexing convention 설명을 Section 3.1 에서제거했다.
8. Table 1 을수정하고 Byte 의 polynomial interpretation 에관한기존 Section 3.2 text 를수정 하여 Section 4 로이동했다.
9. 일반적인 Byte sequence indexing 정의를 Section 3.3 에추가한후 Block example 로 specialization 했으며 Table 2 도수정했다.
10. Section 3.5 heading 을 Word array 중심으로변경하고 notation 을추가했다. State column Word 를 vertical format 으로표현하고 index 설명을개선했다.
11. Finite field 추가정보를위한 reference 를 Section 4 footnote 에넣고 Sections 4.1, 4.2 heading 이 $GF(2^8)$ 을명시하도록수정했다.
12. Section 4.2 를수정해 finite field multiplication 의 explicit general description 을제공했다. 기존 Section 4.2.1 은 xtime 을이용한 illustration 으로대체하여 Section 4.2 에통합했다.

13. Section 4.3 heading 을 fixed matrix multiplication 에 초점을 맞추도록 바꾸고 secondary polynomial-reduction interpretation 을 제거했다. Sections 5.1.3 과 5.3.3 의 MIXCOLUMNS 및 INVMIXCOLUMNS 설명도 이에 맞추어 수정했다.
14. 기존 Section 4.2 에 있던 $GF(2^8)$ multiplicative inverse 설명을 수정하여 새 Section 4.4 로 이동했다.
15. Section 5 의 algorithm component relation 설명을 확장했다. Rijndael parameter 인 Nb 에 대한 간단한 설명을 추가하고 이후 표준에서는 constant value 4 를 직접 사용하도록 했다.
16. CIPHER, Key expansion, inverse cipher pseudocode formatting 을 바꾸고 clarity 를 위해 일부 text 를 수정했다.
17. SHIFTRows 와 INVSHIFTRows 설명을 개선하고 기존 INVSHIFTRows 설명의 오류를 수정했다.
18. AES-128, AES-192, AES-256 KEYEXPANSION illustration 인 새 Figures 6, 7, 8 을 추가했다. Section 5.2 text 도 수정했으며 Round constant 를 명시적으로 표시했다.
19. Equivalent inverse cipher 용 modified Key expansion routine 에 별도 algorithm 을 추가했다. Equivalent inverse cipher 설명은 updated reference 를 인용하는 방향으로 단순화했다.
20. Section 6.2 에 NIST SP 800-133 Rev.2 reference 를 추가했다.
21. Section 6.4 에서 implementation attack discussion 을 확장했다.
22. References section 은 더 이상 appendix 로 label 하지 않는다. Withdrawn publication 을 교체하고 citation information 과 URL 을 수정했다.
23. Appendix C 의 example 을 삭제하고 현재 NIST 에서 관리하는 detailed example vector resource reference 로 대체했다.
24. FIPS 197 Update 1 의 change 를 요약하기 위해 Appendix D 를 새로 만들었다.

번역본끝

이문서는 NIST FIPS 197-upd1 의구조와 technical content 를따라한국어로재작성한비공식 번역본이다. Function name, variable, constant, S-box, equation, hexadecimal value 는 implementation 과원문대조가쉽도록원문 notation 을유지했다.